

Lesson 4 — Comments



  RamRodz Robotics · Team 6366 · Programming Worksheet

Name _____

Date _____

  RAM RODZ ROBOTICS | TEAM #6366 | HILLCREST & WOODMONT HIGH SCHOOLS ·
SIMPSONVILLE, SC

 Focus Check-In:

 Struggling

 Okay

 Ready

TODAY'S BIG QUESTION

If someone else picks up your robot code in three months — or you come back after summer break — how will anyone know what each line is actually supposed to do?

 Parking Lot — Random Thoughts & Questions

1

Key Words — Check Off When Explained

**Comment**

A note written in code that Java ignores when running — it's just for humans to read.

Multi-Line Comment

Starts with `/*` and ends with `*/` — can span as many lines as needed.

Single-Line Comment

Starts with `//` — everything after it on that line is ignored by Java.

Javadoc Comment

Starts with `/**` — a special comment that documents a class or method.

@param

A Javadoc tag that describes one parameter (input) a method receives.

@return

A Javadoc tag that describes what value a method gives back when it finishes.

2

Single-Line Comments — `//`

Listen — Instructor explains what a single-line comment is and when to use it.

Write `//` anywhere on a line. Everything after it is a comment — Java ignores it completely. You can put a comment on its own line, or at the end of a line of code.

Java — Single-Line Comments

```
// This is a comment on its own line — Java skips it entirely

double shooterSpeed = 0.0;    // the shooter starts stopped
boolean isRunning = false;    // shooter is off at startup

// The next line sets the shooter to 80% power
shooterSpeed = 0.8;
isRunning = true;
```

💡 Plain English: **//** tells Java "skip this — it's a note for humans." It works on its own line OR at the end of a code line. Use it to explain what a variable is for, or why a line does what it does.

💡 **Commenting Out Code** — You can also use // to temporarily disable a line of code while testing. Put // in front and Java pretends the line doesn't exist.

Java — Commenting Out Code

```
shooterSpeed = 0.8;           // ← active: using this value
// shooterSpeed = 0.5;        // ← disabled: old value, turned off for now
```

✏️ **Try It — Add a single-line comment to each line below**

```
int intakePort = 2;
```

```
boolean intakeActive = false;
```

☐ I wrote single-line comments on both lines above.

3 Multi-Line Comments — /* */

☐ Listen — Instructor explains multi-line comments and when to use them instead of //.

Start with /* and end with */. Everything between those two markers is a comment — it can span as many lines as you need.

Java — Multi-Line Comment

```
/*  
    Shooter Subsystem – Teleop Control  
    Handles all shooter motor logic during the driver-controlled period.  
    The shooter spins up when the driver holds the fire button.  
    Written by: RamRodz Team 6366  
*/  
  
double shooterSpeed = 0.0;  
boolean isRunning = false;
```

💡 Plain English: **/* opens the comment, */ closes it.** Everything inside is ignored by Java. Use a multi-line comment at the top of a file or code block to describe the big picture — what the code is, why it exists, who wrote it.

Situation	Use This
Explain a single line or variable	//
Describe a block of code (big picture)	/* */
Temporarily disable one line	// in front
Temporarily disable many lines	/* */ around them

Try It — Write a multi-line comment that describes this code block

Code block: `double driveSpeed = 0.0; boolean isMoving = false;`

☐ I wrote a multi-line comment describing the drive block above.





Listen — Instructor explains Javadoc comments and the @param / @return tags.

A Javadoc comment starts with `/**` (two stars). It goes **directly above** a class or method definition and explains what it does, what it needs, and what it gives back. This is the professional standard for documenting Java code — IDEs use it to show hints as you type.

Java — Javadoc on a Method

```
/**
 * Calculates the safe speed for the shooter motor.
 * Clamps the value so the motor never exceeds 90% power.
 *
 * @param requested the speed the driver asked for (0.0 to 1.0)
 * @return          the safe speed to actually send to the motor
 */
static double getSafeSpeed(double requested) {
    if (requested > 0.9) {
        return 0.9; // cap it at 90%
    }
    return requested;
}
```

💡 Plain English: **/** starts the Javadoc, */ ends it.** The first sentence describes what the method does. **@param** explains each input. **@return** explains what comes back. Tools like IntelliJ and VS Code read these to show you helpful hints while you code.

Java — Javadoc on a Class

```
/**
 * Represents the robot's shooter subsystem.
 * Controls motor speed and tracks whether the shooter is running.
 *
 * @author RamRodz — Team 6366
 * @version 1.0
 */
class Shooter {
    double speed;        // current motor speed
    boolean isRunning; // is the shooter on?

    /**
     * Creates a new Shooter with the motor stopped.
     */
    Shooter() {
        speed = 0.0;
        isRunning = false;
    }
}
```

 Plain English: **Put a Javadoc above every class and every method.** On a class, use **@author** to show who wrote it. On a constructor with no parameters, just write a one-sentence description — no @param needed.

Try It — Write a Javadoc comment for this method

Method: `static void setDriveSpeed(double speed)` — sets both drive motors to the same speed, prints a status line.

☐ I wrote a complete Javadoc comment with @param above.

5

All Three Types — Side by Side



Listen — Instructor compares all three comment types and shows the full example.

Java — All Three Comment Types Together

```
/*
  Intake Subsystem — Team 6366
  Controls the intake roller motor during teleop.
*/

/**
 * Starts the intake roller at the given speed.
 *
 * @param speed  roller motor speed from 0.0 (off) to 1.0 (full)
 */
static void startIntake(double speed) {
    double rollerSpeed = speed;    // store the requested speed
    boolean isActive = true;      // mark the intake as running
    System.out.println("Intake ON at: " + rollerSpeed);
}
```

 Plain English: **The multi-line comment describes the file/block. The Javadoc documents the method. The single-line comments explain individual lines.** Each type has its own job — using all three keeps your code professional and easy for teammates to read.

Type	Syntax	Where to Use It
Single-Line	<code>// text</code>	End of a line; explaining one variable; disabling one line
Multi-Line	<code>/* text */</code>	Top of a file or block; big-picture descriptions; disabling many lines
Javadoc	<code>/** text */</code>	Directly above a class or method; always include <code>@param</code> and <code>@return</code>

 Quick Check — Which comment type would you use for each situation?

Explain what a variable named `ballCount` stores:

Document a method named `fire()` that takes a speed parameter:

Write a header at the top of your robot file with your name and the date:

☐ I filled in all three Quick Check answers above.

 Robot Scenario — Your Turn

 Scenario

A teammate wrote the climber subsystem code below — but left **zero comments**. At the next meeting someone else has to fix a bug in it, and nobody knows what anything does.

Your job: add comments to the entire code block so any team member could pick it up and understand it immediately.

- Write a **multi-line comment** at the top of the block describing what this code is
- Write a **Javadoc comment** above the `deployClimber` method (it takes a double position and returns nothing)
- Add a **single-line comment** to every variable line explaining what it stores

Java — The Uncommented Code (read this, then write your comments below)

```
double armPosition = 0.0;
boolean isDeployed = false;
int climberPort = 5;

static void deployClimber(double position) {
    armPosition = position;
    isDeployed = true;
    System.out.println("Climber deployed to: " + armPosition);
}
```

 Write your multi-line comment for the top of the block

 Write your Javadoc for deployClimber

 Write single-line comments for the three variables

```
double armPosition = 0.0;
```

```
boolean isDeployed = false;
```

```
int climberPort = 5;
```

☐

I completed all three comment tasks for the climber code.

☒ **End of Lesson — I Can... (click each one when you're confident)**

☐ Write a single-line comment using `//` to explain what a variable or line of code does.

☐ Write a multi-line comment using `/* */` to describe a block of code or a file header.

☐ Write a Javadoc comment using `/** */` above a method, including `@param` and `@return` tags.

☐ Write a Javadoc comment above a class using `@author`.

☐ Explain why comments matter — for teammates, for future-you, and for professional code.